

# Tratamento de Exceções com **try/catch**

O que é tratamento de exceção?

Tratamento de exceção é uma forma de evitar que o programa seja interrompido quando ocorre um erro.

Em JavaScript utilizamos principalmente:

- **try**
- **catch**
- **finally**

```
try {  
    // código principal  
} catch (erro) {  
    // tratamento do erro  
}
```

# try

O bloco **try** significa:

“Tente executar este código.”

Se ocorrer algum erro dentro dele, o JavaScript interrompe a execução e envia o erro para o **catch**.

# catch

O **catch** captura o erro gerado durante a execução.

# throw

O **throw** é utilizado para criar erros personalizados.

```
const idade = -5;

try {
  if (idade < 0) {
    throw new Error("Idade inválida.");
  }
} catch (erro) {
  console.log(erro.message);
}
```

# finally

O **finally** é um bloco opcional.

Ele sempre será executado:

- Com erro
- Sem erro

É muito utilizado para:

- Fechar conexão com banco de dados
- Encerrar arquivos
- Liberar recursos do sistema

```
try {  
    console.log("Conectando ao banco...");  
    throw new Error("Erro na consulta.");  
} catch (erro) {  
    console.log("Erro:", erro.message);  
} finally {  
    console.log("Fechando conexão com banco.");  
}
```

# EXERCÍCIOS

## Exercício 1 — Divisão Segura

### Enunciado

Desenvolva um programa em Node.js que solicite ao usuário dois números e realize uma divisão entre eles.

O programa deverá:

- Ler dois números digitados pelo usuário
- Realizar a divisão
- Exibir o resultado da operação
- Tratar possíveis erros utilizando **try/catch**

O sistema deve impedir divisão por zero.

## Exercício 2 — Verificador de Idade

### Enunciado

Crie um programa em JavaScript que valide a idade digitada pelo usuário.

O sistema deverá:

- Verificar se o valor é numérico
- Verificar se a idade não é negativa
- Mostrar mensagens de erro quando necessário

## Exercício 3 — Cadastro de Usuário

### Enunciado

Crie um sistema simples de cadastro de usuário em JavaScript.

O sistema deverá solicitar:

- Nome
- Email
- Senha

O programa deverá validar:

- Nome não pode estar vazio
- Email deve conter @
- Senha deve ter no mínimo 6 caracteres

## Exercício 4 — Leitura de Arquivo JSON

### Enunciado

Desenvolva um programa em JavaScript que leia um arquivo chamado **dados.json**.

O sistema deverá:

- Ler o arquivo
- Converter os dados para JSON
- Exibir as informações no terminal
- Tratar possíveis erros

Obs: Deixar o arquivo vazio e rodar o programa e depois colocar o conteúdo:

```
{ } dados.json > ...  
1   {  
2   |  
3   |   "Nome": "Aluno",  
4   |  
5   |   "Idade": "15"  
6   |  
7   }
```

# Exercício 5 — Sistema Bancário Simples

## Enunciado

Crie um sistema bancário simples em JavaScript.

O programa deverá:

- Possuir saldo inicial de R\$1000
- Solicitar valor de saque
- Validar o valor informado
- Atualizar o saldo

O sistema deverá tratar:

- Valores negativos
- Valores maiores que o saldo
- Valores não numéricos